

Gimbal: Adam in a Maximum-Likelihood Kronecker Frame

Tasmai Keni

Abstract

Pending: written after Phase 08 from the claim matrix.

1 Introduction

SOAP [33] preconditions each weight matrix by running Adam in a rotated frame (Q_L, Q_R) , the eigenbasis of Shampoo’s Kronecker factors, and it is one of the strongest optimizers for language-model pre-training at moderate scale [30, 34]. Inside SOAP two estimators answer to two different models. Adam’s second moment is the maximum-likelihood estimate of a free array of variances in the frame; the frame comes from pooled factors $\mathbb{E}[GG^\top]$, $\mathbb{E}[G^\top G]$, statistics of a Kronecker-*product* covariance, whose variance array is separable. The two coincide only when the variances are separable and the factors’ eigenvalues are distinct.

This paper asks what the frame should be under SOAP’s own model, and whether the answer helps. The answer to the first question is a classical object: the frame that maximizes the likelihood of the free-diagonal model makes the rotated second moment as diagonal as possible in the log-determinant sense, a joint-diagonalization problem from statistics and blind source separation. We derive its score and Fisher information, and an online estimator, *Gimbal*, that follows it with one damped natural-gradient step on $O(m) \times O(n)$ per block of steps, using only matrix products after a short warm start.

Contributions.

- The objective behind SOAP’s preconditioner family (the profile KL of proposition 1) and an identifiability and efficiency theory for frame estimators: the likelihood identifies every pair of rows whose variance profiles differ, where pooled factors need the profile *sums* to differ, and every single-factor estimator, SOAP’s and KL-Shampoo’s included, loses statistical efficiency except in special cases (section 4); 53 statements are machine-checked in Lean.
- Gimbal (section 3): a likelihood frame flow with empirical-Bayes variance shrinkage toward the separable fit, an innovation-centred statistic, a pooled warm start and adaptive amortization; matrix products only after step 50.
- Evidence at three levels, including where it is negative: Monte Carlo on gradient streams with known truth (section 5), where Gimbal’s frame is several times more accurate than SOAP’s and KL-SOAP’s; a small language model (section 6), where the premise that real gradient variances are non-separable fails, Gimbal beats SOAP narrowly and KL-SOAP beats Gimbal; and a 125M-parameter, 2.5B-token comparison with AdamW and SOAP on a TPU v4-32 under equal tuning budgets (sections 7 and 8).

At 125M parameters the predeclared decision rule gives: Gimbal beats AdamW on loss: **[pending]**; beats SOAP on loss: **[pending]**; the headline claim (beats both on loss and is no slower than SOAP per step) is **[pending]**. Gimbal’s step is 1.9% slower than SOAP’s on the TPU.

2 Background: SOAP and the frames of matrix preconditioners

Let $W \in \mathbb{R}^{m \times n}$ be a weight matrix with stochastic gradient G . Kronecker-factored preconditioners keep two factor matrices $L \approx \mathbb{E}[GG^\top]$ and $R \approx \mathbb{E}[G^\top G]$ (Shampoo [2, 13]). SOAP [33] observed that Shampoo with power 1/2 is Adafactor run in the eigenbasis of (L, R) and replaced Adafactor by Adam: with Q_L, Q_R the eigenvectors of L, R and rotated coordinates $Z_U(X) = Q_L^\top X Q_R$, SOAP runs Adam on $Z_U(G)$ and rotates the step back,

$$\Delta W = -\eta Q_L \left(\hat{M} \oslash (\sqrt{\hat{V}} + \epsilon) \right) Q_R^\top, \quad (1)$$

where $\hat{M}, \hat{V}$ are Adam’s bias-corrected moments of $Z_U(G)$. The frame is refreshed every f steps by one power-iteration step and a QR decomposition, $Q \leftarrow \text{qr}(LQ)$; $\hat{V}$ is re-ordered with it. SOAP is among the strongest optimizers for language-model pre-training at small and medium scale [30, 33, 34], and a line of work has improved its frame: the KL-optimal factor rule of KL-Shampoo and KL-SOAP [20], refreshes that include the current gradient and run every step for stability at scale [18], eigenvalue corrections that remove grafting [7], and cheaper or one-sided rotations [11, 22].

The model behind each part of SOAP. Adam’s diagonal second moment is the maximum-likelihood estimate of a free array of variances $D \in \mathbb{R}_{>0}^{m \times n}$ in the frame: it treats the entries of $Z_U(G)$ as independent with $\mathbb{E}[Z_U(G)^{\circ 2}] = D$. Call this model $\text{KRD}(U, D)$ (Kronecker-rotated diagonal). The pooled factors L, R , on the other hand, are moment statistics of a different model, a Kronecker *product* covariance, in which $D = \lambda \mu^\top$ is separable (Shampoo, K-FAC; KL-Shampoo replaces them by that model’s two-sided maximum-likelihood rule). SOAP therefore estimates its eigenvalues under one model and its frame under another. Lin et al. noted that SOAP’s D cannot be a Kronecker product [20]; they did not derive a frame estimator for the free-diagonal model. This paper does.

3 Gimbal: the maximum-likelihood frame, followed online

3.1 The objective behind SOAP’s own preconditioner

For a frame $U = (Q_L, Q_R) \in O(m) \times O(n)$ write $d_U = \mathbb{E}[Z_U(G)^{\circ 2}]$ and let S be the second-moment operator of $\text{vec } G$. The Kullback–Leibler projection of $\mathcal{N}(0, S)$ onto SOAP’s family, minimized over D , is

$$J(U) = \min_{D>0} \text{KL}(\mathcal{N}(0, S) \parallel \text{KRD}(U, D)) = \frac{1}{2} \left(\sum_{ij} \log(d_U)_{ij} - \log \det S \right) \geq 0, \quad (2)$$

attained at $D = d_U$ (Proposition 1). The best member of SOAP’s preconditioner family uses the frame that makes the rotated second moment “as diagonal as possible” in this log-determinant sense: a joint-diagonalization (common principal components) problem [9, 27], here restricted to Kronecker frames. Adam’s average of $Z_U(G)^{\circ 2}$ estimates d_U for whatever frame is used; the frame is the part SOAP takes from another model.

3.2 The likelihood score and its Fisher information

Perturb the left frame as $Q_L \mapsto Q_L(I + \Omega)$ with Ω skew-symmetric. With $A = D^{\circ -1}$, the score of the working Gaussian log-likelihood of one gradient is the skew-symmetric matrix

$$E_L = S_L - S_L^\top, \quad S_L = Z(Z \odot A)^\top, \quad (3)$$

and on the right $E_R = S_R - S_R^\top$ with $S_R = Z^\top(Z \odot A)$ (Lemma 3). In the true frame its Fisher information is diagonal in the rotation coordinates,

$$F_{ik}^L = \sum_j \left(\frac{D_{ij}}{D_{kj}} + \frac{D_{kj}}{D_{ij}} - 2 \right) = (DA^\top + AD^\top)_{ik} - 2n, \quad (4)$$

so the rotation of the pair (i, k) is identifiable exactly when the rows $D_{i\cdot}$ and $D_{k\cdot}$ differ, while pooled factors need their *sums* to differ (Theorem 4).

3.3 The flow

Gimbal follows the maximum-likelihood frame online with one damped natural-gradient step on $O(m) \times O(n)$:

$$\Omega_L = -\alpha_t \frac{S_L - S_L^\top}{F_L + \delta n}, \quad Q_L \leftarrow \text{polish}(Q_L(I + \Omega_L + \frac{1}{2}\Omega_L^2)), \quad (5)$$

with entrywise division, the diagonal of Ω_L set to zero, entries clipped to $[-\theta_{\max}, \theta_{\max}]$ and the spectral norm capped at one; the right side is the mirror image. The retraction $I + \Omega + \frac{1}{2}\Omega^2$ loses orthogonality only at fourth order and a Newton–Schulz polish restores it (Theorem 12); no QR or eigendecomposition is needed. Dividing by F is Amari’s natural gradient [1], so near the optimum every pair contracts at the same rate, independent of its eigen-gap (Theorem 6). The rate follows the bias-corrected schedule $\alpha_t = \alpha/(1 - (1 - \alpha)^t)$, which makes the frame a bias-corrected exponential average of per-gradient estimates (Theorem 7), the identity that justifies Adam’s bias correction.

The variances that weight the score. D in (3) is an exponential average of $Z^{\circ 2}$ with the frame’s own memory $\beta_D = 1 - \alpha$, the profile likelihood with one forgetting factor (Lemma 8), shrunk toward its separable (additive-in-log) fit by a positive-part James–Stein factor whose noise level is measured from an interleaved odd/even split of the average (Proposition 9, after the core shrinkage of Hoff et al. [14]). With separable statistics the weights become KL-Shampoo’s efficient ones; with strongly non-separable statistics they become the free maximum-likelihood variances.

The innovation. A gradient with a non-zero mean tilts the uncentred statistic away from the true frame (Proposition 11); Gimbal fits the frame to the innovation $G_t - c_t \hat{M}_{t-1}$, where $\hat{M}_{t-1}$ is the previous bias-corrected momentum and c_t its own James–Stein factor, $c_t \approx 0$ for zero-mean gradients and $c_t \rightarrow 1$ when a deterministic mean dominates.

Warm start and amortization. The frame starts from the eigenvectors of the first gradient’s factors and, after $T_w = 50$ steps, restarts from the eigenvectors of the pooled factors of those steps, a one-step estimator in Le Cam’s sense (Remark 10): these are the only two eigendecompositions. The score is accumulated over blocks of k_t steps and the frame moves once per block, so the retraction, polish and Fisher matrix are paid once per block. The block length is adaptive, $k_t = \text{clamp}(\text{round}(K\alpha/\alpha_t), 1, K)$ with $K = 4$: the amortized step matches the per-step flow only to first order in $k\alpha_t$, so the early steps, where α_t is large, are not amortized.

Defaults. $\alpha = 1 - \beta_2 = 0.05$ (the frame, its variances and Adam’s second moment share one estimation window of effective size 39, SOAP’s convention for its factors), $\delta = 0.003$, $\theta_{\max} = 0.25$, $K = 4$, $T_w = 50$, $\beta = (0.9, 0.95)$; in every comparison β_1 is the same for all frame-based methods (SOAP’s 0.95 at 125M, section 7). Figure 1 summarizes one step.

Algorithm 1: one Gimbal step for a matrix W with gradient G (step t , rate η_t)

1. $Z \leftarrow Q_L^\top G Q_R$ (rotated gradient)
2. $c \leftarrow$ James-Stein factor of $\hat{M}_{t-1}$; $\tilde{Z} \leftarrow Z - c \hat{M}_{t-1}^{\text{rot}}$ (innovation, Prop. 11)
3. Adam in the frame: $M^{\text{rot}} \leftarrow \beta_1 M^{\text{rot}} + (1 - \beta_1) Z$, $V \leftarrow \beta_2 V + (1 - \beta_2) Z^{\circ 2}$;
 $W \leftarrow (1 - \eta_t \lambda) W - \eta_t Q_L (\hat{M}^{\text{rot}} \oslash (\sqrt{\hat{V}} + \epsilon)) Q_R^\top$
4. $V^F \leftarrow (1 - \alpha) V^F + \alpha \tilde{Z}^{\circ 2}$ (and its odd-step half); $D \leftarrow$ shrunk variances (Prop. 9)
5. accumulate the scores $S_L - S_L^\top$, $S_R - S_R^\top$ of (3) at $\tilde{Z}$, D
6. every k_t steps: $\Omega \leftarrow$ (5) with the mean score and the block's rate; $Q \leftarrow \text{polish}(Q(I + \Omega + \frac{1}{2}\Omega^2))$;
re-express M^{rot} in the new frame
7. at $t = T_w$: restart the frame from the pooled factors' eigenvectors; transport V , V^F (Theorem 14)

Figure 1: Gimbal's step. Everything after the warm start is matrix products and elementwise work.

Cost. Per step Gimbal rotates the gradient, rotates the step back and forms the score: three products of cost $m^2n + mn^2$, the same count as SOAP's rotations and factor updates. Once per block it adds the Fisher matrix, the retraction and the polish, $O(m^3 + n^3)$ products in float32. SOAP adds a QR decomposition every $f = 10$ steps. Gimbal's state is two frames, the momentum, Adam's second moment, the flow's variance average and its odd half, plus the accumulated scores between moves.

4 Theory

Every result carries an evidence label: [L1] observed numerically, [L2] conjectured, [L3] proved under additional assumptions, [L4] proved as stated, [L5] machine-checked in Lean 4 with Mathlib (53 theorems; every one depends only on the axioms `propext`, `Classical.choice` and `Quot.sound`; appendix B maps statements to Lean names). Proofs are in appendix A and in the project's theory document. The Gaussian likelihood is a working likelihood that fixes an estimating equation; no claim that gradients are Gaussian is made.

Proposition 1 (Profile KL; [L4], per coordinate [L5]). *For every frame U and $S \succ 0$,*

$$\min_{D \succ 0} \text{KL}(\mathcal{N}(0, S) \parallel \text{KRD}(U, D)) = J(U)$$

with J of (2), attained at $D = d_U$, and $J(U) = 0$ iff $S = C(U, d_U)$.

Proposition 2 (What pooled factors estimate; [L4]). *If $S = C(U^*, D)$ then $\mathbb{E}[GG^\top] = Q_L^* \text{diag}(r) Q_L^{*\top}$ with $r_i = \sum_j D_{ij}$ and $\mathbb{E}[G^\top G] = Q_R^* \text{diag}(c) Q_R^{*\top}$ with $c_j = \sum_i D_{ij}$. Pooled factors are consistent only for pairs whose row (column) sums differ; with tied sums the cost of an arbitrary choice within the tied plane is unbounded (Example: D with rows $(a, b), (b, a)$; [L5]).*

Lemma 3 (Score; [L4]). *At the true frame the partial derivatives of the negative log-likelihood of one observation with respect to the rotation coordinates are the entries of E_L and E_R in (3); both are skew-symmetric.*

Theorem 4 (Fisher information and identifiability; [L4], finite-sum statements [L5]). *In $\text{KRD}(U^*, D)$ the Fisher information of one observation is diagonal in the rotation coordinates, block-orthogonal to D , with entries (4). The rotation of the pair (i, k) is locally identifiable iff $D_{i\cdot} \neq D_{k\cdot}$; pooled factors identify it iff $\sum_j D_{ij} \neq \sum_j D_{kj}$. The first condition is strictly weaker.*

Theorem 5 (Efficiency; item 1 [L3], items 2–4 [L5] except one direction of item 3, [L4]). *Consider frame estimators from a weighted factor $\hat{L}_w = \frac{1}{N} \sum_t G_t Q_R^* \text{diag}(w) Q_R^{*\top} G_t^\top$ (SOAP: $w \equiv 1$; KL-Shampoo: $w = 1/\hat{\mu}$).*

1. $\sqrt{N} \hat{\theta}_{ik}^w$ is asymptotically normal with variance $V_w = \sum_j w_j^2 D_{ij} D_{kj} / (\sum_j w_j (D_{ij} - D_{kj}))^2$.
2. $V_w \geq 1/F_{ik}$, with equality iff $w_j \propto D_{kj}^{-1} - D_{ij}^{-1}$.
3. One weight vector is efficient for all pairs iff $D^{\circ-1} = \mathbf{1}\beta^\top + \alpha w^\top$. Under separability $D = \lambda \mu^\top$ this holds with $w = 1/\mu$ (KL-Shampoo is efficient exactly under separability); SOAP's $w \equiv 1$ then loses the factor $n/n_{\text{eff}}(\mu)$.
4. There are non-separable arrays on which every single positive weighting is strictly inefficient for several pairs at once; the likelihood score applies pair-dependent weights and attains $1/F$.

Theorem 6 (Flow: fixed points, rate, noise; item 1 [L5], item 2 [L3], item 3 [L5]). (1) *Frames without within-column cross moments are fixed points of the mean flow.* (2) *Near U^* the undamped mean step contracts every pair by $1 - \alpha$, independent of the eigen-gap; with damping by $1 - \alpha F / (F + \delta n)$.* (3) *The linearized recursion has stationary variance $\frac{\alpha}{2-\alpha} \frac{1}{F}$: the Cramér–Rao variance at an effective sample size $(2 - \alpha)/\alpha$.*

Theorem 7 (Bias-corrected schedule; [L5]). *With $a_t = (1 - \beta)/(1 - \beta^t)$ the recursion $\theta_{t+1} = \theta_t + a_{t+1}(x_t - \theta_t)$ is the bias-corrected exponential average of its inputs.*

Lemma 8 (Variances of the flow; item 1 [L5], item 2 identity [L5], expansion [L3]). (1) *With forgetting weights the weighted likelihood is maximized over D by the bias-corrected average of $Z^{\circ 2}$ with the same coefficient: the frame and its variances share one forgetting factor.* (2) *Plug-in weights w cost exactly $V_w = 1/F + \|w_\perp\|^2 / \langle w, w^* \rangle^2$, $w_\perp \perp w^*$; for noisy variance estimates with relative variance $\bar{\varepsilon}^2$ the inflation is $1 + \bar{\varepsilon}^2 K/F + O(\bar{\varepsilon}^4)$ with an explicit second-order term.*

Proposition 9 (Empirical-Bayes variances; risk identity [L5], noise estimate [L3]). *Among $A + c\hat{R}$ (A the additive fit of $\log V^F$, $\hat{R} = R + E$ the residual) the risk $(c - 1)^2 \|R\|^2 + c^2 \nu$ is minimized exactly at $c^* = \|R\|^2 / (\|R\|^2 + \nu)$. Gimbal uses the positive-part plug-in with ν from an odd/even split; separable arrays give $c \rightarrow 0$ (KL-Shampoo's efficient weights), strongly non-separable ones $c \rightarrow 1$ (free maximum likelihood). The shrinkage changes the variance of the frame estimate, never its mean.*

Remark 10 (Warm start as a one-step estimator; weighting [L5], rest [L3]). *Restarting from pooled factors at T_w and continuing the bias-corrected schedule gives the restart value exactly the weight $1 - \beta^{T_w}$ that the first T_w per-sample estimates would have had; this is Le Cam's one-step construction (a consistent start followed by Fisher scoring).*

Proposition 11 (Frame statistic of a gradient with a mean; items 1–2 [L4], optimal factor [L5], plug-in [L3]). *If $G_t = \mu_t + N_t$ with predictable mean, the expected skew score of the uncentred statistic at the true frame is $\Theta(\Theta \odot A)^\top - (\Theta \odot A)\Theta^\top \neq 0$ for $\Theta = Z_{U^*}(\mu)$; subtracting $c\hat{M}_{t-1}$ scales this bias by $(1 - c)^2$ at a variance cost $c^2 \eta \text{tr} \Sigma$, minimized at $c^* = \|\mu\|^2 / (\|\mu\|^2 + \eta \text{tr} \Sigma)$ with $\eta = (1 - \beta_1)(1 + \beta_1^{t-1}) / ((1 + \beta_1)(1 - \beta_1^{t-1}))$.*

Theorem 12 (Retractions; [L5]). *For skew Ω : $(I - \Omega)^{-1}(I + \Omega)$ is orthogonal; $X = I + \Omega + \frac{1}{2}\Omega^2$ satisfies $X^\top X = I + \frac{1}{4}\Omega^4$; one Newton–Schulz step on $X^\top X = I + E$ leaves $-\frac{3}{4}E^2 + \frac{1}{4}E^3$.*

Theorem 13 (Equivariance; one-step algebra [L5]). *Running Gimbal on $PG_t R^\top$ from (PQ_L, RQ_R) reproduces every rotated quantity and gives $\Delta W' = P\Delta W R^\top$. The initialization by eigenvectors is equivariant up to their signs, to which the update is invariant once every quantity computed from Ω is invariant under $\Omega \mapsto S\Omega S$; the trust region's power iteration is started from the largest column of*

Ω for this reason (an earlier fixed start vector made the update depend on the eigensolver’s sign conventions).

Theorem 14 (Transport; [L5]). *If the frame moves by an orthogonal P , a diagonal second moment d seen in the new frame is $(P^{\circ 2})^\top d$; $P^{\circ 2}$ is doubly stochastic, and for Kronecker frames $(P_L \otimes P_R)^{\circ 2} = P_L^{\circ 2} \otimes P_R^{\circ 2}$.*

Theorem 15 (Descent and scale invariance; [L5]). *With $\beta_1 = 0$, $\langle G, U \rangle = \sum_{ij} Z_{ij}^2 / (\sqrt{\hat{V}_{ij}} + \epsilon) \geq 0$. Replacing every gradient by cG leaves the frame flow unchanged.*

Conjecture 16 (Global convergence; [L2], numerical support [L1]). *For KRD statistics with pairwise distinct profiles the minimizers of J are U^* up to signed permutations and every other critical point is a strict saddle. From Haar-random frames the noise-free flow reached the global minimum from all 4,500 of 4,500 starts, and constructed critical points were strict saddles that the flow escaped.*

5 Monte Carlo evidence on synthetic gradient streams

Before any language model was trained, the method was tested where the truth is known: gradient streams drawn from $\text{KRD}(U^*, D)$ with controlled non-separability γ (with $\log D$ the sum of an additive part and γ times an interaction), spectral slope, tied profiles, drifting frames and heavy tails. Every peer ran on the same streams at its best memory and at a matched effective sample size, on ten paired seeds (table 1; protocol, gates and every table in `experiments/phase2/report.md`).

Frame efficiency. Measured by the frame KL J of (2), the default’s frame is better than every peer’s in every non-separable cell. Pooled across cells (random effects), SOAP’s frame KL is 25.5 times Gimbal’s, SOAP real-time’s 24.9 times and KL-SOAP’s 16.2 times. In separable cells, where KL-factor weights are themselves efficient (Theorem 5.3), the worst ratio of Gimbal to the best peer is 1.093 (bootstrap upper bound 1.110, within the predeclared 25% margin). Tied row sums, which pooled factors cannot resolve, are identified by the likelihood (consistency over a 20-fold longer horizon), drifting frames are tracked with lower error, and Student- t noise does not reverse any ordering.

Optimization on noisy quadratics. On quadratics whose curvature and gradient noise share a KRD structure, with every method’s learning rate tuned on the same grid and evaluated on fresh seeds, the default reaches the lowest mean final loss in every configuration (table 1; in the separable high-noise configuration not every paired difference is significant); the gate passes. This gate failed once, before the frame statistic was centred on the innovation (Proposition 11): the deterministic part of a quadratic’s gradient tilted the uncentred statistic, exactly as item 2 of that proposition predicts.

Theory against simulation. The predicted frame KL in the linear regime, the plug-in inflation of Lemma 8 (after a second-order correction that a failed pre-registered check made necessary), the split-sample noise estimate, the one-step contraction of Theorem 6 and the Hessian of J agree with simulation within their predeclared tolerances. The cost model (section 7) and the numerical checks (float32 vs float64, orthogonality over 10^4 steps, scale invariance, degenerate inputs) pass.

γ	σ	Gimbal loss	/AdamW	/SOAP	/SOAP-rt	/KL-SOAP	/Muon	lowest
0	0.1	0.0316	0.488	0.810	0.752	0.936	0.697	yes
0	1	1.3153	0.878	0.993	0.868	0.999	0.928	yes
1	0.1	0.0344	0.479	0.783	0.756	0.846	0.666	yes
1	1	1.0461	0.887	0.951	0.802	0.958	0.885	yes
2	0.1	0.0241	0.468	0.781	0.715	0.754	0.601	yes
2	1	0.5055	0.905	0.929	0.920	0.939	0.895	yes

Table 1: Noisy quadratics (E2.5): Gimbal’s mean final loss and its geometric-mean ratio to each peer on paired evaluation seeds (below 1: Gimbal lower), learning rates tuned per method on equal grids. γ : non-separability of the curvature; σ : noise level.

What the synthetic evidence does not show. The streams were drawn from the model whose likelihood Gimbal maximizes, with non-separability set by hand. Whether real gradients have non-separable variances, and whether a better frame translates into lower language-model loss, are separate questions, taken up next.

6 A small language model: the premise, a failure and its repair

A byte-level Llama-style model ($d = 128$, four layers, sequence length 128, batch 32, 800 steps) on a FineWeb-Edu sample gives a cheap test on real gradients. Every optimizer received the same four-point learning-rate grid (factor two, extended at an edge) on seed 0 and was then run on two more seeds at its selected rate. Nine optimizers, and Gimbal with a frame move at every step, were compared in PyTorch.

The default first lost to SOAP. Table 2 lists the outcome. The default of the time (amortized frame moves every four steps, rotation rate $\alpha = 0.02$) lost to SOAP on every seed; moving the frame every step won on every seed; KL-SOAP was best. Selection runs on separate seeds isolated two causes. The amortized move matches the per-step flow only to first order in $k\alpha_t$, and with the bias-corrected schedule α_t is large early: the gap to the per-step flow was largest at step 100 and never closed. And the 100-step frame memory of $\alpha = 0.02$, chosen on stationary synthetic streams, lagged the drifting statistics of a language model: larger α was better at every amortization. SOAP’s memory knob received the same selection budget. The repair fixed $\alpha = 1 - \beta_2$ (one estimation window for the frame and Adam’s second moment, SOAP’s own convention) and the adaptive amortization of section 3, chosen by a rule written before the TPU timings were known: faster-moving variants were better on this model but cost more step time than the rule allowed (section 7). Every Monte Carlo gate was re-run with the new default and still passes.

Confirmation on fresh seeds. With the learning rate re-selected on seed 20 and seeds 21–22 added, the repaired default is lower than SOAP and AdamW on every seed: mean 1.4929 versus 1.4952 (SOAP) and 1.6671 (AdamW). The margin over SOAP is small (mean difference -0.0023, smallest -0.0001; one-sided paired t -test with Holm correction $p = 0.087$). On the same seeds KL-SOAP reaches 1.4829, better than Gimbal on every seed, and SOAP real-time 1.4948, indistinguishable from Gimbal.

The premise does not hold at this scale. At steps 100, 400 and 800, 96 independent minibatch gradients of eight matrices were collected at fixed weights, frames were fitted on half of them and

optimizer	lr	mean	seeds 0, 1, 2
gimbal	0.004	1.5058	1.5118, 1.5002, 1.5052
gimbal-k1	0.004	1.4934	1.4951, 1.4908, 1.4944
soap	0.004	1.4970	1.4990, 1.4959, 1.4962
soap-rt	0.004	1.4927	1.4944, 1.4918, 1.4919
klsoap	0.004	1.4828	1.4873, 1.4789, 1.4823
adamw	0.001	1.6633	1.6472, 1.6809, 1.6620
muon	0.01	1.5136	1.5160, 1.5136, 1.5113
normuon	0.005	1.5184	1.5191, 1.5154, 1.5207
splus	0.05	1.6412	1.6393, 1.6391, 1.6451
aro	0.004	1.5171	1.5181, 1.5149, 1.5184

Table 2: Small language model (E3.2), final validation loss at each optimizer’s selected learning rate. “gimbal” is the default before the repair of this section (fixed amortization, $\alpha = 0.02$); “gimbal-k1” moves the frame every step. Other peers: SOAP real-time, KL-SOAP, Muon, NorMuon, SPlus, ARO (appendix C).

scored on the other half by the held-out likelihood. The likelihood frame beats SOAP’s pooled frame in 48 of 48 matrices (median gain 1,897 nats of held-out log-likelihood). But the non-separability index of the variances, the share of $\text{Var}(\log D)$ not explained by the additive fit, has median 0.005 (largest 0.021), far below the 0.05 that the predeclared premise required: the rotated variances of this model are almost exactly separable. The likelihood frame’s advantage over SOAP is therefore the efficiency gain of Theorem 5.3 on steep separable spectra, which KL-factor estimators share: against KL frames the likelihood frame wins in only 27 of 48 matrices (median 9 nats). This is the ordering the theory predicts when $\kappa \approx 0$, and it matches the losses above. The hypothesis’ own kill criterion ($\kappa \approx 0$ and SOAP real-time at least as good as Gimbal) is met at this scale.

7 Experimental setup at 125M parameters

Model and data. A Llama-style decoder [32]: 12 layers, width 768, 12 heads, SwiGLU hidden width 2048, rotary position embeddings, pre-norm RMSNorm, no biases, tied input and output embeddings, vocabulary 50,304, context 1024; 123.6M parameters. Training data are the first 2.5B tokens of the shuffled training documents of FineWeb-Edu **sample-10BT** [26], tokenized with the GPT-2 tokenizer; a fixed hash of the document identifier assigns 20M-token validation and test splits from disjoint documents (checksums in the repository). As a check of the data and loss pipeline, the OpenAI GPT-2 (124M) checkpoint [29] scores 3.262 nats on our validation split, against 3.2924 reported on the FineWeb-Edu validation shard of build-nanogpt [17].

Training. 240 sequences of 1024 tokens per step (245,760 tokens): confirmatory runs take 10,172 steps (2.49987B tokens, every training sequence once), tuning runs 2,500 steps (614M tokens). The learning rate warms up linearly for 5% of the steps and decays by a cosine to 10% of its peak; gradients are clipped at global norm 1; decoupled weight decay 0.1 applies to the hidden matrices only. The hidden matrices use the optimizer under test; embeddings and norms use AdamW ($\beta = (0.9, 0.95)$, no weight decay) with the same peak learning rate and schedule as the hidden matrices (the AdamW baseline is one optimizer, so its tuned β_2 applies to every parameter). bfloat16 compute, float32 master weights and optimizer state. A seed fixes the initialization and the data order and is shared by all optimizers.

optimizer	seed	val. loss	SE	val. ppl	test loss	step (ms)	spikes	max $\ g\ $
-----------	------	-----------	----	----------	-----------	-----------	--------	-------------

Table 3: Confirmatory runs (seeds 2 and 3): final validation loss with its block-bootstrap standard error, validation perplexity, test loss (read once, after the decision), steady-state step time, loss spikes and largest gradient norm after step 500.

Implementation on a TPU v4-32. All three optimizers are written in JAX [3] as pure per-matrix steps, verified against the PyTorch references (per-step agreement to about 10^{-12} in float64 for Gimbal; SOAP against the official implementation). The hidden matrices are stored in two buckets of equal shape (48 of 768×768 and 36 of 768×2048); the optimizer state of each bucket is sharded across the 16 chips, each chip updates its own matrices with batched products, and only the parameter increments are gathered. Every optimizer gets the same layout, batching, sharding and precision (all optimizer products in float32). Attention uses the Pallas flash-attention kernel [5]: with 512×512 tiles a training gradient step takes 0.170 s against 0.208 s with XLA attention and 0.259 s with the kernel’s default tiles.

Step time. Measured as the median, over 20-step blocks, of the mean step time in a block (so that SOAP’s refresh every 10 steps and Gimbal’s frame move every four are included), the training step takes 162.0 ms with AdamW, 167.0 ms with SOAP and 170.2 ms with Gimbal, 1.9% more than SOAP. Gimbal’s frame move on the 2048-dimensional side (retraction, orthogonality check and Newton–Schulz polish in float32) is the difference; moving the frame every two steps or every step would cost 5.0% and 11.0% more than SOAP.

Tuning (equal budgets). Each optimizer received the same budget: seven peak learning rates on a factor-two grid around its reference default (AdamW 10^{-3} , SOAP and Gimbal $3 \cdot 10^{-3}$), two seeds each, at the 614M-token horizon, followed by its most sensitive secondary knob at the selected rate (AdamW $\beta_2 \in \{0.95, 0.98, 0.999\}$, SOAP’s factor memory $\beta_s \in \{0.9, 0.95, 0.99\}$, Gimbal’s rate $\alpha \in \{0.025, 0.05, 0.1\}$), two seeds each. Because every selection fell on an edge of its three values, each optimizer received one more value by a rule fixed before the second seed was seen ($\beta_2 = 0.9$, $\beta_s = 0.8$, $\alpha = 0.2$). Selection used the full validation split; the test split was not read. Gimbal runs with SOAP’s momentum $\beta_1 = 0.95$, so that the two differ only in the frame. Its first tuning trials used Adam’s $\beta_1 = 0.9$ and were unstable around the end of the warm-up at learning rates $\geq 1.5 \cdot 10^{-3}$; short diagnostic runs showed that SOAP is unstable with $\beta_1 = 0.9$ as well, and that neither moment transport nor per-step frame moves remove the instability (appendix E). Those trials are reported but were not used: Gimbal was re-tuned with the full budget. Selected values: AdamW learning rate 0.002 with $\beta_2 = 0.95$; SOAP 0.012 with $\beta_s = 0.9$; Gimbal 0.006 with $\alpha = 0.2$ (appendix D). Every learning rate is inside its grid; Gimbal’s α is not: a faster frame (shorter memory) was better at every step of its grid, so larger values remain untested.

8 Results at 125M parameters and 2.5B tokens

Tuning. Figure 2 shows the mean final validation loss of the two tuning seeds against the peak learning rate. Selected: AdamW 0.002 ($\beta_2 = 0.95$), SOAP 0.012 ($\beta_s = 0.9$), Gimbal 0.006 ($\alpha = 0.2$).

Confirmatory runs. Table 3 lists every run.

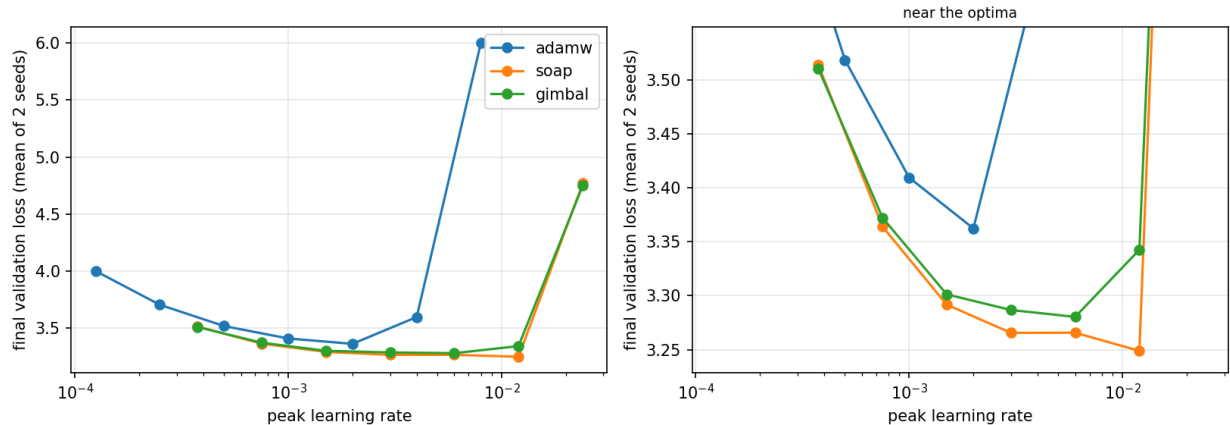


Figure 2: Tuning (614M tokens, mean of seeds 0 and 1): final validation loss against the peak learning rate (Gimbal with SOAP’s momentum, after the repair of appendix E).

c	seed	Gimbal $- c$	paired SE	tokens to c ’s loss (B)	multiplier	Gimbal min	c min
-----	------	--------------	-----------	---------------------------	------------	------------	---------

Table 4: Gimbal against each competitor c , paired by seed: difference of final validation loss with its paired bootstrap standard error, tokens Gimbal needs to reach c ’s final loss on the evaluation curve, the token multiplier (full budget over those tokens), and wall-clock to that point and c ’s full wall-clock (steady-state step time $\times$ steps).

Mechanism diagnostics.

9 Related work

Kronecker-factored preconditioners. Shampoo [2, 13] and K-FAC [24] precondition with Kronecker factors of the gradient’s second moment or of the Fisher information. SOAP [33] runs Adam in Shampoo’s eigenbasis. KL-Shampoo and KL-SOAP [20] derive the factor update from a KL objective for a Kronecker-product covariance and observe that SOAP’s diagonal cannot be such a product; Gimbal instead estimates the frame under the free-diagonal model SOAP’s Adam already assumes. Purifying Shampoo [7] corrects stale and mis-scaled eigenvalues and adapts the refresh frequency; the NVIDIA study [18] shows that a stale frame causes instabilities at large batch and refreshes it every step with the current gradient included; COSMOS [22] applies SOAP on a leading subspace and Muon elsewhere; ARO [11] chooses a one-sided rotation by a norm-informed Procrustes policy; SPlus [10] uses the sign of the momentum in a stale Shampoo frame; DyKAF [35] computes the Kronecker factors of the Fisher information with projector-splitting integrators; and EVON [25] runs a diagonal variational method in SOAP’s eigenspace. Apart from ARO, all of these take their frame from Kronecker-product statistics. Everett and Qiu [8] find that the optimal optimizer memory grows with the training horizon, which is consistent with the shorter frame memory our small model preferred.

Orthogonalized updates. Muon [16, 21] orthogonalizes the momentum by Newton–Schulz iterations and NorMuon [19] adds neuron-wise second moments. Gimbal also uses Newton–Schulz iterations, but only to keep its frames orthogonal.

[pending]

Figure 3: Validation loss on the 5.24M-token evaluation subset against tokens and against wall-clock (steady-state step time $\times$ steps); solid seed 2, dashed seed 3.

[pending]

Figure 4: Gimbal’s runs: noise-corrected non-separability index of the flow’s variances (median over the matrices of each bucket; the predeclared premise threshold 0.05 dashed) and the empirical-Bayes shrinkage factor c .

Benchmarks. Recent controlled comparisons [30, 34] find matrix optimizers ahead of AdamW at small scale by a margin that shrinks with model size, and the ranking among them sensitive to tuning and horizon; we follow their emphasis on equal tuning budgets.

Where the estimator comes from. The maximum-likelihood frame of a family of covariance matrices that share eigenvectors is the common-principal-components problem [9] and, in signal processing, joint approximate diagonalization [27]. Gimbal’s score is the skew part of the EASI relative-gradient update [4] with a heteroscedastic Gaussian score, its Fisher normalization is Amari’s natural gradient [1], and its identifiability condition is the Pham–Cardoso condition for separating non-stationary sources [28], with time blocks replaced by the other matrix mode. Shrinking a matrix-variate covariance toward its separable (Kronecker) part by an empirical-Bayes factor is the core shrinkage of Hoff et al. [14]; centring a second moment on the gradient’s running mean is centred RMSProp [12] and AdaBelief [36], here applied only to the frame’s statistic and with a James–Stein factor [15]. A search of the literature through 2026-10-07 found no optimizer that estimates its frame by this likelihood (appendix F).

10 Limitations

- *Scale and seeds.* One model size (125M), one data set, one token budget (about one Chinchilla), two seeds per optimizer in the confirmatory runs; the decision rule is built for that design, and differences below the seed-to-seed spread cannot be resolved.
- *Competitors.* The TPU study compares AdamW, SOAP and Gimbal only (compute constraint). On the small model KL-SOAP beat Gimbal and SOAP real-time tied it; neither was run at 125M, nor were Muon and the other peers.
- *Horizon.* Learning rates and secondary knobs were tuned at a quarter of the confirmatory horizon for every optimizer alike; optimal values can drift with the horizon.
- *Premise.* The likelihood frame is most valuable when the rotated variances are non-separable. On real gradients of the small model they were almost exactly separable, and the diagnostics of the 125M runs (section 8) show where the larger model stands.
- *Cost.* On the TPU v4 Gimbal’s float32 frame moves make its step slower than SOAP’s; the comparison of step times is specific to this hardware, its software stack and our implementation of both optimizers.
- *Theory.* The efficiency and rate results are local or asymptotic ([L3]) and the global convergence of the flow is a conjecture.

11 Reproducibility statement

Code, configurations, raw logs and the scripts that generate every number and figure of this paper are in the repository (README.md). The Lean development builds with `cd formal && lake build`. The Monte Carlo study re-runs with the scripts under `experiments/phase2` and regenerates its report with `make_report.py`; the small language model with `experiments/phase3`; the TPU benchmarks with `benchmarks/tpu` (docs/performance.md); tuning and confirmatory runs with `scripts/tpu/tune.py` and `scripts/tpu/main_runs.py` from frozen code snapshots (each run records its commit); the analysis with `python analysis/phase08.py`; and this paper with `make -C paper`. Data: FineWeb-Edu sample-10BT at a pinned revision, tokenized by `scripts/data/prepare_fineweb_edu.py` (SHA-256 of every split in `data/tokens_manifest.json`). Every failure, refuted statement and design change is recorded with its mechanism in `research/ledger/`. The project was carried out by an AI agent (Claude, Anthropic) following the written protocol in `phases/`, under the author’s direction.

References

- [1] Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2): 251–276, 1998.
- [2] Rohan Anil, Vineet Gupta, Tomer Koren, Kevin Regan, and Yoram Singer. Scalable second order optimization for deep learning. *arXiv preprint arXiv:2002.09018*, 2020.
- [3] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. <http://github.com/jax-ml/jax>, 2018.
- [4] Jean-François Cardoso and Beate Hvam Laheld. Equivariant adaptive source separation. *IEEE Transactions on Signal Processing*, 44(12):3017–3030, 1996.
- [5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, 2021.
- [7] Runa Eschenhagen, Aaron Defazio, Tsung-Hsien Lee, Richard E. Turner, and Hao-Jun Michael Shi. Purifying Shampoo: Investigating Shampoo’s heuristics by decomposing its preconditioner. *arXiv preprint arXiv:2506.03595*, 2025.
- [8] Katie Everett and Shikai Qiu. Optimizer memory schedules for outscaling the overtraining axis. *arXiv preprint arXiv:2609.04577*, 2026.
- [9] Bernhard N. Flury. Common principal components in k groups. *Journal of the American Statistical Association*, 79(388):892–898, 1984.
- [10] Kevin Frans, Sergey Levine, and Pieter Abbeel. A stable whitening optimizer for efficient neural network training. *arXiv preprint arXiv:2506.07254*, 2025.

- [11] Wenbo Gong, Javier Zazo, Qijun Luo, Puqian Wang, James Hensman, and Chao Ma. ARO: A new lens on matrix optimization for large models. *arXiv preprint arXiv:2602.09006*, 2026.
- [12] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [13] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *International Conference on Machine Learning*, 2018.
- [14] Peter Hoff, Andrew McCormack, and Anru R. Zhang. Core shrinkage covariance estimation for matrix-variate data. *arXiv preprint arXiv:2207.12484*, 2022.
- [15] W. James and Charles Stein. Estimation with quadratic loss. In *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 361–379, 1961.
- [16] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024.
- [17] Andrej Karpathy. build-nanogpt. <https://github.com/karpathy/build-nanogpt>, commit 6104ab1, 2024.
- [18] Mikail Khona, Aditya Vavre, Boxiang Wang, Deyu Fu, Hao Wu, Mike Chrzanowski, Bryan Catanzaro, Dheevatsa Mudigere, Jeff Pool, Michael Lightstone, Mohammad Shoeybi, Mostofa Patwary, Nima Tajbakhsh, and Tijmen Blankevoort. SOAP, Muon, and beyond: Pushing LLM pretraining scales. *arXiv preprint arXiv:2607.20548*, 2026.
- [19] Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. NorMuon: Making Muon more efficient and scalable. *arXiv preprint arXiv:2510.05491*, 2025.
- [20] Wu Lin, Scott C. Lowe, Felix Dangel, Runa Eschenhagen, Zikun Xu, and Roger B. Grosse. Understanding and improving Shampoo and SOAP via Kullback-Leibler minimization. *arXiv preprint arXiv:2509.03378*, 2025.
- [21] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, et al. Muon is scalable for LLM training. *arXiv preprint arXiv:2502.16982*, 2025.
- [22] Liming Liu, Zhenghao Xu, Zixuan Zhang, Hao Kang, Zichong Li, Chen Liang, Weizhu Chen, and Tuo Zhao. COSMOS: A hybrid adaptive optimizer for memory-efficient training of LLMs. *arXiv preprint arXiv:2502.17410*, 2025.
- [23] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- [24] James Martens and Roger Grosse. Optimizing neural networks with Kronecker-factored approximate curvature. In *International Conference on Machine Learning*, 2015.
- [25] Adrian Robert Minut, Nico Daheim, Marco Miani, Mohammad Emtiyaz Khan, Wu Lin, and Thomas Möllenhoff. SOAP-bubbles: Structured weight uncertainty for neural networks. *arXiv preprint arXiv:2606.23357*, 2026.
- [26] Guilherme Penedo, Hynek Kydliček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

- [27] Dinh-Tuan Pham. Joint approximate diagonalization of positive definite Hermitian matrices. *SIAM Journal on Matrix Analysis and Applications*, 22(4):1136–1152, 2001.
- [28] Dinh-Tuan Pham and Jean-François Cardoso. Blind separation of instantaneous mixtures of nonstationary sources. *IEEE Transactions on Signal Processing*, 49(9):1837–1848, 2001.
- [29] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners, 2019. OpenAI technical report.
- [30] Andrei Semenov, Matteo Pagliardini, and Martin Jaggi. Benchmarking optimizers for large language model pretraining. *arXiv preprint arXiv:2509.01440*, 2025.
- [31] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, 2020.
- [32] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [33] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing Shampoo using Adam. *arXiv preprint arXiv:2409.11321*, 2024.
- [34] Kaiyue Wen, David Hall, Tengyu Ma, and Percy Liang. Fantastic pretraining optimizers and where to find them. *arXiv preprint arXiv:2509.02046*, 2025.
- [35] Nikolay Yudin, Ekaterina Grishina, Andrey Veprikov, Alexandr Beznosikov, and Maxim Rakhuba. DyKAF: Dynamical Kronecker approximation of the Fisher information matrix for gradient preconditioning. *arXiv preprint arXiv:2511.06477*, 2025.
- [36] Juntang Zhuang, Tommy Tang, Yifan Ding, Sekhar Tatikonda, Nicha Dvornek, Xenophon Papademetris, and James S. Duncan. AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients. In *Advances in Neural Information Processing Systems*, 2020.

A Proofs

Complete proofs, the proof audit (quantifiers, divisions, degenerate cases) and the history of every revised statement are in `theory/gimbal_theory.md`; the main steps are as follows.

Proposition 1. With $C = (Q_L \otimes Q_R) \text{diag}(\text{vec}D)(Q_L \otimes Q_R)^\top$, $\text{KL} = \frac{1}{2}[\text{tr}(C^{-1}S) - mn + \log \det C - \log \det S]$ separates over coordinates, each minimized at $D_{ij} = (d_U)_{ij}$; non-negativity of the remainder is Hadamard’s inequality applied to the rotated S , whose diagonal is d_U .

Lemma 3 and Theorem 4. Under $Q_L \mapsto Q_L(I + \Omega)$, $Z \mapsto (I - \Omega)Z$ to first order, which gives the score. Its second moment at the true frame keeps only the terms $\mathbb{E}[Z_{ij}^2 Z_{kj}^2] = D_{ij} D_{kj}$, giving F_{ik}^L ; products of scores of different pairs, or with a variance score, contain an odd power of an entry and vanish. Each summand of F_{ik}^L vanishes iff $D_{ij} = D_{kj}$.

Theorem 5. Item 1 is the delta method for the eigenvector perturbation of $\hat{L}_w$ (the off-diagonal entry over the eigen-gap). Item 2 is Cauchy–Schwarz with $r_j = w_j(D_{ij} - D_{kj})$, $r_j^2 = f_j g_j$. Item 3 follows by fixing one row; item 4 by a 3×2 array whose two row pairs demand incompatible weights.

Theorem 6. The expected score vanishes when cross moments vanish; near the optimum it equals the Fisher information times the displacement, and the natural-gradient step divides it out; item 3 is the AR(1) recursion.

Proposition 9 and 11. Both risk identities are $(c - 1)^2 S + c^2 \nu$, minimized at $S/(S + \nu)$; for the innovation, $E[W(W \odot A)^\top]$ is diagonal for noise W with independent entries, so only the mean’s rank-one term survives in the skew part.

Theorems 12–15 are algebraic identities, checked in Lean.

B Machine-checked statements

Lean 4 [6] with Mathlib [31]; `lake build in formal/`; the audit prints the axioms of every listed theorem (all 53 use only `propext`, `Classical.choice` and `Quot.sound`; no `sorry`). Statement labels follow the numbering of the theory document.

statement	Lean names	file
Thm 3.2	<code>weighted_cs</code>	<code>Efficiency.lean</code>
Thm 3.2	<code>weighted_ge_mle</code>	<code>Efficiency.lean</code>
Thm 3.2 (SOAP)	<code>pooled_ge_mle</code>	<code>Efficiency.lean</code>
Thm 3.4	<code>mle_weights_attain</code>	<code>Efficiency.lean</code>
Thm 3.3 (KL)	<code>kl_weights_efficient_separable</code>	<code>Efficiency.lean</code>
Thm 3.3 (SOAP)	<code>pooled_loss_separable, pooled_loss_ge_one</code>	<code>Efficiency.lean</code>
Thm 3.3 (suff.)	<code>efficient_if_additive_inverse,</code> <code>efficient_if_additive_inverse’, variance_scale_invariant</code>	<code>Efficiency.lean</code>
Thm 3.4 (strict)	<code>factor_estimators_strictly_inefficient</code>	<code>Efficiency.lean</code>
Thm 2	<code>fisher_pos_iff, fisher_nonneg</code>	<code>Identifiability.lean</code>
Thm 2	<code>pooled_implies_profile</code>	<code>Identifiability.lean</code>
Thm 2 (strict)	<code>tie_but_identifiable</code>	<code>Identifiability.lean</code>
Example 1	<code>tie_cost_pos, tie_cost_unbounded</code>	<code>Identifiability.lean</code>
Thm 5.1	<code>cayley_orthogonal, one_sub_skew_det_ne_zero,</code> <code>dotProduct_skew_self</code>	<code>Retraction.lean</code>
Thm 5.2	<code>expm2_defect</code>	<code>Retraction.lean</code>
Thm 5.3	<code>ns_error</code>	<code>Retraction.lean</code>
Thm 7	<code>hadamard_sq_kronecker</code>	<code>Transport.lean</code>
Thm 7	<code>rowsum_hadamard_sq, colsum_hadamard_sq</code>	<code>Transport.lean</code>
Thm 7	<code>transport_preserves_total</code>	<code>Transport.lean</code>
Thm 6	<code>rotated_coords_invariant, update_covariant</code>	<code>Equivariance.lean</code>
Thm 8.1	<code>frobenius_adjoint, descent, descent_strict</code>	<code>Equivariance.lean</code>
Thm 8.2	<code>generator_scale_invariant, fisher_scale_invariant</code>	<code>Equivariance.lean</code>
§5	<code>fisher_term_identity</code>	<code>Flow.lean</code>
Thm 2	<code>score_variance_term</code>	<code>Flow.lean</code>
Thm 4.1	<code>stationary_generator</code>	<code>Flow.lean</code>
Thm 4.3	<code>variance_recursion_closed_form, variance_recursion_tendsto</code>	<code>Flow.lean</code>
Thm 4.4	<code>bias_corrected_schedule</code>	<code>Flow.lean</code>
Prop. 1	<code>eigenvalue_cost_nonneg, eigenvalue_cost_eq_iff</code>	<code>Flow.lean</code>
Lemma 5.4.1	<code>weighted_profile_variance, weighted_profile_variance_eq_iff</code>	<code>Variance.lean</code>
Lemma 5.4.2	<code>excess_variance_identity, excess_component_orthogonal</code>	<code>Variance.lean</code>
Prop. 5.5	<code>shrinkage_risk_ge, shrinkage_risk_eq_iff</code>	<code>Variance.lean</code>
Remark 5.6	<code>restart_closed_form</code>	<code>Variance.lean</code>
Prop. 5.7	<code>ema_weight_sq_sum’ (with ‘shrinkage_risk_eq_iff’)</code>	<code>Variance.lean</code>
§3 (E2.10 a)	<code>pair_rotation_product, pair_rotation_cost_nonneg,</code> <code>pair_rotation_cost_le, pair_rotation_cost_sum_le</code>	<code>PairCost.lean</code>
§3 (E2.10 a)	<code>sin_sq_mul_cos_sq_le, pair_rotation_cost_le_half_fisher_sq</code>	<code>PairCost.lean</code>
Prop. 9	<code>gimbal_state_le_soap</code>	<code>PairCost.lean</code>

Not formalized: the delta-method asymptotics (Theorem 5.1), the Fisher identity at a displaced parameter (Theorem 6.2), Hadamard’s inequality (cited), the second-order expansion of Lemma 8, the statistical properties of the split-sample noise estimate, and the conjecture; these are proved on paper or checked by Monte Carlo.

C Optimizers and hyper-parameters

Exact update rules, defaults and sources of AdamW [23], SOAP and Gimbal, and of the PyTorch-only peers of section 6 (SOAP real-time, KL-SOAP, Muon, NorMuon, SPlus, ARO), are in `docs/optimizers.md`. Gimbal’s defaults: $\beta = (0.9, 0.95)$ ($\beta_1 = 0.95$, SOAP’s, at 125M), $\epsilon = 10^{-8}$, $\alpha = 0.05$, $\delta = 0.003$, relative floor 10^{-8} , $\theta_{\max} = 0.25$, spectral cap 1, $K = 4$ (adaptive), $T_w = 50$. SOAP: $\beta = (0.95, 0.95)$, factor memory $\beta_s = \beta_2$, $f = 10$. AdamW: $\beta = (0.9, 0.95)$. Weight decay 0.1 on hidden matrices for all three.

D Tuning runs

Every Phase 06 trial (2,500 steps, 614M tokens; final loss on the full validation split):

optimizer	peak lr	secondary	seed	final validation loss
adamw	0.000125	default	0	3.9960
adamw	0.000125	default	1	4.0058
adamw	0.00025	default	0	3.7018
adamw	0.00025	default	1	3.7076
adamw	0.0005	default	0	3.5140
adamw	0.0005	default	1	3.5225
adamw	0.001	default	0	3.4041
adamw	0.001	default	1	3.4144
adamw	0.002	default (replicate)	0	3.3617
adamw	0.002	b2=0.98	0	3.3643
adamw	0.002	b2=0.98	1	3.3602
adamw	0.002	b2=0.999	0	3.4009
adamw	0.002	b2=0.999	1	3.4152
adamw	0.002	b2=0.9	0	3.3597
adamw	0.002	b2=0.9	1	3.3657
adamw	0.002	default	0	3.3617
adamw	0.002	default	1	3.3627
adamw	0.004	default	0	3.5972
adamw	0.004	default	1	3.5973
adamw	0.008	default	0	5.9846
adamw	0.008	default	1	6.0234
gimbal	0.000375	default	0	3.5041
gimbal	0.000375	default	1	3.5166
gimbal	0.00075	default	0	3.3697
gimbal	0.00075	default	1	3.3746
gimbal	0.0015	default	0	3.3016
gimbal	0.0015	default	1	3.3007
gimbal	0.003	default	0	3.2836
gimbal	0.003	default	1	3.2897
gimbal	0.006	rot_rate=0.025	0	3.2805
gimbal	0.006	rot_rate=0.025	1	3.2888
gimbal	0.006	rot_rate=0.1	0	3.2724
gimbal	0.006	rot_rate=0.1	1	3.2675
gimbal	0.006	rot_rate=0.2	0	3.2636
gimbal	0.006	rot_rate=0.2	1	3.2657
gimbal	0.006	default	0	3.2817
gimbal	0.006	default	1	3.2789
gimbal	0.012	default	0	3.3596
gimbal	0.012	default	1	3.3253
gimbal	0.024	default	0	4.7383
gimbal	0.024	default	1	4.7604
soap	0.000375	default	0	3.5110
soap	0.000375	default	1	3.5167
soap	0.00075	default	0	3.3626
soap	0.00075	default	1	3.3653
soap	0.0015	default	0	3.2925
soap	0.0015	default	1	3.2907
soap	0.003	default	0	3.2641
soap	0.003	default	1	3.2669
soap	0.006	default	0	3.2647
soap	0.006	default	1	3.2666
soap	0.012	default	0	3.2484
soap	0.012	default	1	3.2492
soap	0.012	shampoo_beta=0.8	0	3.2534
soap	0.012	shampoo_beta=0.8	1	3.2502
soap	0.012	shampoo_beta=0.99	0	3.2642
soap	0.012	shampoo_beta=0.99	1	3.2685
soap	0.012	shampoo_beta=0.9	0	3.2469
soap	0.012	shampoo_beta=0.9	1	3.2453
soap	0.024	default	0	4.7785
soap	0.024	default	1	4.7602

E Diagnosis of an early instability

In the first tuning trials Gimbal used Adam’s momentum $\beta_1 = 0.9$ while SOAP used its official 0.95. At peak learning rates $\geq 1.5 \cdot 10^{-3}$ Gimbal’s gradient norm rose to 5–20 and its loss increased for about 40 steps around the end of the warm-up; the runs recovered but finished about 0.03 nats behind SOAP. Before any configuration was frozen, the tuning sweep was paused and the following runs were made at learning rate $3 \cdot 10^{-3}$, seed 0, for the first 300 of the 2,500 tuning steps (same schedule), changing one setting at a time; the repair (SOAP’s β_1 for Gimbal) was fixed before the last three were seen (decision D-008 in the repository).

run	max $\ g\ $, steps 80–159	training loss, steps 280–299
Gimbal, $\beta_1 = 0.9$ (before C-019)	20.1	4.318
SOAP, $\beta_1 = 0.95$ (official)	1.5	4.141
Gimbal, warm-start frames only (no moves)	1.8	4.760
Gimbal, $\beta_1 = 0.95$	2.1	4.193
Gimbal, rotation per move ≤ 0.1	1.8	4.630
Gimbal, frame move every step	15.8	4.207
Gimbal, moments transported at moves	22.5	4.330
SOAP, $\beta_1 = 0.9$	74.4	4.392

The instability needs the lower momentum: it disappears with $\beta_1 = 0.95$, and SOAP is unstable with $\beta_1 = 0.9$ too. Freezing or slowing the frame also removes it, but trains much more slowly. Transporting the second moments to the new frame at every move (theorem 14) does not remove it, so stale second moments are not the cause. A float64 comparison of the distributed training step with a per-matrix loop rules out an implementation error.

F Novelty check

Searches of arXiv, OpenReview and conference proceedings (2026-10-06, repeated on the final method on 2026-10-07; the queries and hits are listed in `research/literature/frontier.md`) for optimizers that estimate a Kronecker frame by joint diagonalization, common principal components, EASI-type relative gradients or Pham–Cardoso non-stationary source separation; for empirical-Bayes shrinkage of second moments toward Kronecker-separable structure in optimizers; and for centred frame statistics. No overlapping method was found; closest are KL-Shampoo/KL-SOAP [20] (Kronecker-product likelihood), TurboSoap (a blog post tracking the factors’ eigenvectors by matrix products), core shrinkage [14] (statistics, not optimization) and Stein-rule gradient shrinkage. Patents and non-English venues were not searched; “apparently distinct” is the strongest statement this supports.